

SORBONNE UNIVERSITÉ

MASTER AI2D

Rapport de projet PAI2D : Programmation de comportements pour le robot Pogobot-wheel

Auteurs :

Rania MOSBAH,
Noriane LABIAD

Encadrants :

Loona MACABRE,
Nicolas BREDECHE

Date :

20 mai 2026

Lien GIT :

<https://github.com/nana-08/pogobot-size-estimation>

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectif	2
2	État de l’art	3
2.1	Présentation du système	3
2.1.1	Description du robot	3
2.1.2	Architecture logicielle	3
2.1.3	Les LEDs	3
2.2	Présentation du code de Loona Macabre	6
2.2.1	Structure générale	6
2.2.2	Fonctions	7
3	Contributions	7
3.1	Premier livrable : Reproduction d’expériences	7
3.1.1	Résultats	8
3.2	Second livrable : Mode de diffusion en parallèle	8
3.2.1	Stratégie	8
3.2.2	Mise en place (modifications du code)	8
3.2.3	Résultats	8
3.3	Troisième livrable : Mode de diffusion en séquence	10
3.3.1	Stratégie	10
3.3.2	Mise en place (modifications du code)	10
3.3.3	Résultats	10
3.4	Quatrième livrable : Mécanisme de consensus et de convergence	11
3.4.1	Stratégie	12
3.4.2	Mise en place (modifications du code)	13
3.4.3	Résultats	14
4	Conclusion	15
4.1	Bilan	15
4.2	Limites	15
4.3	Perspectives	15
5	Annexes	16
5.1	Cahier des charges	16
5.1.1	Contexte et définition du problème	16
5.1.2	Objectifs fonctionnels	16
5.1.3	Contraintes techniques et matérielles	16
5.1.4	Critères d’acceptation	17
5.1.5	Livrables attendus (code)	17
5.1.6	Environnement d’exécution	17
5.2	Manuel d’utilisation	17

1 Introduction

1.1 Contexte

Les robots **Pogobot** sont des robots mobiles de petite taille (6 cm), équipés de roues et se déplaçant à une vitesse d'environ 20 cm/sec. Ils sont capables d'évoluer de manière autonome et d'échanger des informations avec leurs voisins immédiats via communication infrarouge. Leur conception compacte et leur communication locale en font une plateforme particulièrement adaptée à l'étude des systèmes multi-robots.

Ces robots s'inscrivent dans le domaine de la robotique en essaim, où un ensemble d'agents simples interagit localement pour produire un comportement collectif. Dans ce type d'architecture distribuée, aucune entité centrale ne coordonne le système : chaque robot agit uniquement à partir des informations disponibles dans son voisinage.

La mobilité des robots et la nature locale des échanges rendent le réseau dynamique et évolutif. Les interactions varient en fonction des déplacements, ce qui constitue un cadre pertinent pour l'étude d'algorithmes distribués appliqués à des systèmes autonomes.

Dans le cadre de notre projet, les expérimentations ont été réalisées sur des robots **Pogobot** sans roues, donc immobiles. Les déplacements n'étaient donc pas pris en compte dans nos tests, ce qui signifie que le voisinage entre robots restait globalement stable au cours des expériences. Cette configuration nous a permis de nous concentrer principalement sur les mécanismes de diffusion, de synchronisation et de consensus, sans introduire les variations de connectivité liées à la mobilité des robots.

1.2 Objectif

L'objectif de ce projet est de programmer ces robots pour réaliser plusieurs tâches typiques de la robotique autonome et collective. Plus précisément, il s'agit d'implémenter et d'évaluer des mécanismes de diffusion d'information au sein d'un essaim de robots mobiles, afin de permettre le comptage distribué, la mise à jour dynamique des estimations et l'atteinte d'un consensus global.

Le projet vise ainsi à étudier le comportement du système lorsque plusieurs diffusions sont exécutées en parallèle ou en séquence, ainsi qu'à analyser les conditions nécessaires pour garantir la convergence vers une valeur commune. L'enjeu principal est d'assurer la robustesse et la fiabilité du système face aux variations de paramètres, au nombre d'itérations et aux éventuelles défaillances de robots.

2 État de l'art

2.1 Présentation du système

2.1.1 Description du robot

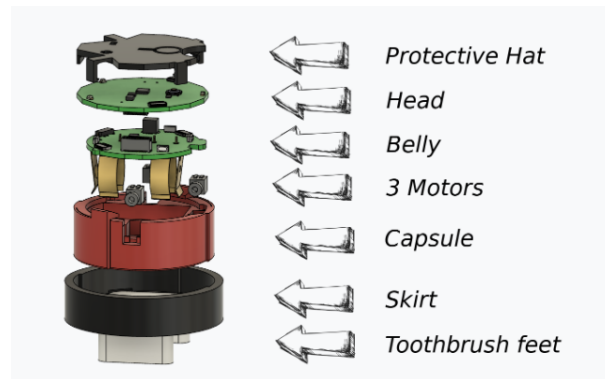


FIGURE 1 – Composants du robot

Le robot est composé de :

- photosensors
- 3 contrôleurs de moteurs, des cartes électroniques qui servent d'intermédiaire entre le cerveau du robot et le moteur
- IMU (Inertial Measurement Unit), un capteur électronique qui mesure le mouvement et l'orientation d'un robot
- communication infrarouge, un moyen d'échanger des données sans fil en utilisant la lumière infrarouge
- batterie longue durée

2.1.2 Architecture logicielle

Le robot dispose d'une architecture logicielle modulaire, organisée en trois niveaux principaux :

- Firmware bas-niveau : contrôle direct des moteurs et capteurs (IMU, IR, LED).
- API haut-niveau : fonctions pour déplacer le robot, lire les capteurs et communiquer entre robots.
- Comportements utilisateur : code définissant les actions autonomes et la logique de navigation. S'exécute au-dessus de l'API et peut être simulé ou testé sur des robots physiques.

Communication et simulation :

Communication interne : protocole IR pour échanges entre robots.

Simulation : un simulateur logiciel reproduit l'API du robot pour tester les comportements avant déploiement réel.

2.1.3 Les LEDs

Les LEDs jouent un rôle essentiel dans la visualisation de l'état interne des robots. Elles permettent d'observer en temps réel l'évolution des estimations calculées par chaque

robot sans avoir besoin d'utiliser les traces affichées sur le terminal.

Chaque robot possède une valeur de concentration c , mise à jour au cours des échanges avec ses voisins. À partir de cette concentration, le robot calcule une estimation du nombre total de robots grâce à la formule :

$$\text{estimation} = \text{round} \left(\frac{1}{c} \right)$$

Ainsi, plus la concentration est faible, plus l'estimation du nombre de robots est élevée. Cette estimation entière est ensuite convertie en couleurs affichées par les LEDs du robot.

Le système d'affichage repose sur deux groupes de LEDs :

- une LED arrière utilisée pour afficher le chiffre des dizaines :
`pogobot_led_setColors(..., 0);`
- une LED supérieure utilisée pour afficher le chiffre des unités :
`pogobot_led_setColors(..., 3).`

L'estimation est donc décomposée en dizaines et unités, chaque chiffre étant représenté par une couleur spécifique.

Ce système permet donc d'afficher des estimations allant de 0 à 99 à l'aide de deux couleurs simultanées.

Par exemple :

- une estimation égale à 5 affiche une LED jaune ;
- une estimation égale à 42 affiche une LED orange pour les dizaines (4) et une LED magenta pour les unités (2) ;
- une estimation égale à 10 affiche une LED rouge pour les dizaines et une LED noire (éteinte) pour les unités ;
- une estimation supérieure ou égale à 100 affiche une LED bleue afin d'indiquer un dépassement de capacité d'affichage.

Lorsqu'un robot reçoit une information provenant d'un voisin, sa concentration est mise à jour puis les LEDs changent automatiquement de couleur en fonction de la nouvelle estimation calculée. Cette représentation visuelle permet d'observer facilement la propagation d'une diffusion dans l'essaim ainsi que la convergence progressive des robots vers une valeur commune.

La communication entre robots est réalisée grâce aux émetteurs et récepteurs infrarouges intégrés. Chaque robot échange uniquement avec les robots présents dans son voisinage immédiat, ce qui reproduit le fonctionnement d'un réseau distribué local. Les déplacements des robots modifient dynamiquement les connexions entre voisins et influencent directement la circulation des informations dans le système.

Afin de préparer les expériences, les robots sont programmés et alimentés via une station de connexion USB permettant le chargement des batteries et le téléversement du code. Une fois déconnectés, les robots exécutent leur programme de manière autonome.

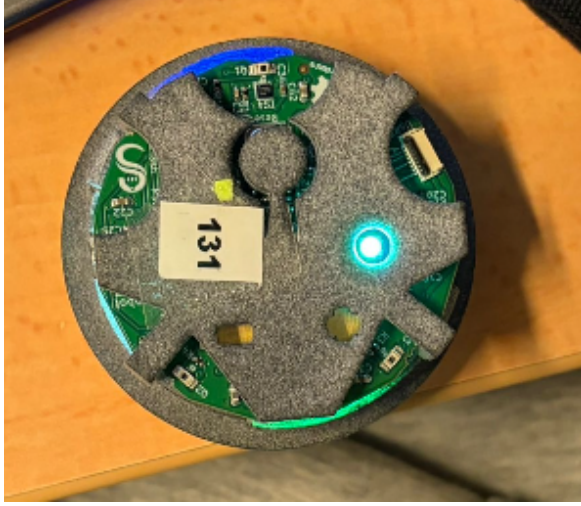


FIGURE 2 – Led bleue, concentration = 0.0, correspondant à une estimation supérieure ou égale à 100



FIGURE 3 – Led rouge, concentration = 1.0, correspondant à une estimation égale à 1

Les couleurs des LEDs correspondent aux chiffres affichés sur les LEDs des dizaines et des unités :

Chiffre	Couleur associée
0	Noir (LED éteinte)
1	Rouge
2	Magenta
3	Rose
4	Orange
5	Jaune
6	Vert
7	Mint
8	Cyan
9	Bleu clair

Chaque chiffre correspond indirectement à une plage de concentration puisque :

$$c \approx \frac{1}{\text{estimation}}$$

Les principales correspondances sont les suivantes :

- Rouge : estimation = 1, donc $0.67 < c \leq 1$
- Magenta : estimation = 2, donc $0.40 < c \leq 0.67$
- Rose : estimation = 3, donc $0.286 < c \leq 0.40$
- Orange : estimation = 4, donc $0.222 < c \leq 0.286$
- Jaune : estimation = 5, donc $0.182 < c \leq 0.222$
- Vert : estimation = 6, donc $0.154 < c \leq 0.182$
- Mint : estimation = 7, donc $0.133 < c \leq 0.154$
- Cyan : estimation = 8, donc $0.118 < c \leq 0.133$
- Bleu clair : estimation = 9, donc $0.105 < c \leq 0.118$
- Bleu : estimation ≥ 100

2.2 Présentation du code de Loona Macabre

2.2.1 Structure générale

Structure des messages :

Les messages sont transmis en infrarouge. Chaque message contient :

```
typedef struct RawMessage {
    int robot_id; //identifiant unique de l'émetteur
    uint8_t msg_type; //type du message (HELLO, ACK ou OK)
    int recipient_id; //destinataire visé (-1 pour diffusion)
    int hello_msg_id; //numéro séquentiel du message HELLO
    uint8_t probe_attempt; //compteur de tentatives (mode debug)
    int partner_msg_id; //ID du message auquel on répond
    double concentration_value; //valeur de concentration de l'émetteur
} RawMessage;
```

Structure des robots

```
typedef struct connecting_robot {
    int robot_id; // ID du robot partenaire
    int msg_id; // ID du message envoyé
    int partner_msg_id; // ID du message du partenaire
    double concentration_value; // concentration du partenaire
} connecting_robot;
```

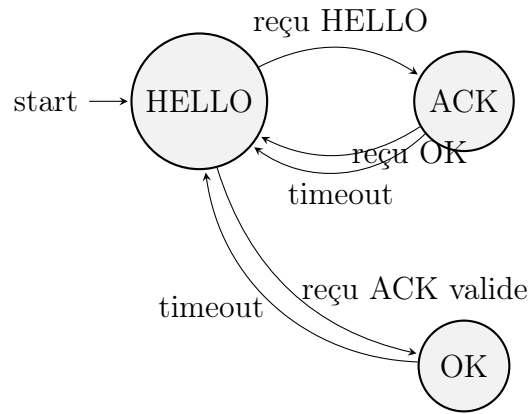
Cette structure stocke l'état de la communication en cours.

Les trois types de messages

- **HELLO** : message de diffusion initiale. Un robot envoie sa concentration à tout voisin.
- **ACK** : accusé de réception. Réponse à un HELLO, contient la concentration du répondeur.
- **OK** : confirmation finale. Termine l'échange et déclenche la mise à jour.

Machine à états du protocole Chaque robot peut être dans l'un des trois états d'envoi suivants :

1. **État HELLO** (`sending_HELLO = 1`) :
 - Aucun partenaire de handshake actif
 - Le robot diffuse périodiquement sa concentration
 - En recevant un HELLO d'un voisin, il passe en état ACK
2. **État ACK** (`sending_ACK = 1`) :
 - Un partenaire est enregistré (`my_partner`)
 - Le robot répond avec ACK contenant sa concentration
 - Timeout de $T_{\max} = 50$ steps : abandon et retour HELLO
3. **État OK** (`sending_OK = 1`) :
 - Le robot a reçu un ACK valide
 - Il envoie OK pour confirmer l'échange
 - Timeout de 50 steps : validation forcée



2.2.2 Fonctions

Manipulation des nombres

- `log_10(power)` : retourne l'exposant d'une puissance de 10 (ex : $1000 \rightarrow 3$)
- `round_double(d)` : arrondit un réel à l'entier le plus proche
- `double_get_integer_part(value)` : extrait la partie entière d'un réel
- `double_get_decimal_part(value)` : extrait la partie décimale (ex : $3.14 \rightarrow 14$)
- `double_get_decimal_part_string(value, res)` : convertit la partie décimale en chaîne de caractères

Communication

- `send_message(...)` : construit et envoie un message infrarouge (HELLO, ACK ou OK)

Gestion du handshake

- `clear_connecting_neighbour(robot)` : réinitialise la structure du partenaire (ID = -1, concentration = 0)

Visualisation

- `set_rgb_colors(concentration)` : convertit la concentration en code couleur RGB et allume les LED (dizaine = tête, unité = queue)

3 Contributions

3.1 Premier livrable : Reproduction d'expériences

Pour ce premier livrable, il s'agit de reproduire des expérimentations existantes réalisées par Loona Macabre sur plateformes mobiles, en faisant varier certains paramètres clés, notamment le pourcentage de concentration attribuées initialement à chaque robot. Ces variations permettront d'analyser l'influence de la répartition initiale sur la vitesse de convergence et sur la précision du décompte final. En complément, les expériences réalisées par Loona à l'aide de robots mobiles seront répliquées avec un nombre d'itérations plus élevé, afin d'évaluer la robustesse et la stabilité des résultats dans la durée.

3.1.1 Résultats

Ce livrable n'a pas pu être réalisé car il n'y avait pas assez de robots pogobot-wheels disponibles au laboratoire.

3.2 Second livrable : Mode de diffusion en parallèle

Ce livrable consiste à exécuter plusieurs algorithmes de diffusion simultanément. Le système développé permet l'exécution concurrente de plusieurs diffusions, chacune utilisant une valeur différente du paramètre D (nombre de bonds ou portée de diffusion). Et l'observation de l'impact du parallélisme sur la cohérence des estimations.

L'objectif sous-jacent est de déterminer si l'exécution parallèle permet l'obtention d'une estimation fiable sans générer d'interférences entre les diffusions.

3.2.1 Stratégie

Notre stratégie s'appuie sur le code principal, qui utilise une probabilité pour la distribution de la concentration, et qui partage ses informations ainsi que sa concentration via des messages. Pour réaliser des diffusions en parallèle, il serait nécessaire d'émettre plusieurs messages simultanément, ce qui impliquerait de dupliquer la méthode de diffusion au sein du code. Autrement dit, chaque diffusion parallèle nécessite sa propre instance du mécanisme de diffusion, avec ses propres paramètres (notamment la valeur de D) et ses propres canaux de communication, afin d'éviter les interférences entre les flux d'information.

3.2.2 Mise en place (modifications du code)

Pour mettre en œuvre ce mécanisme, nous avons procédé aux modifications suivantes :

- Ajout d'un nouveau paramètre `KAPPA2` (qui correspond à la valeur de D dans le code), fixé à 0.2. Ce choix permet aux diffusions de varier tout en convergeant vers la même valeur.
- Ajout d'une nouvelle variable `concentration_value2` dans la structure du robot, dédiée au calcul de la concentration en parallèle.
- Modification de la signature et de la configuration des messages afin qu'ils puissent transporter simultanément les informations relatives aux deux diffusions.

En pratique, nous avons largement dupliqué le code existant pour gérer indépendamment chaque diffusion parallèle.

3.2.3 Résultats

Nous avons testé notre algorithme sur 5 robots, avec un robot ayant ces deux concentrations initiales à 1.0 et les 4 autres à 0.0.

```

PogoBoot> serialboot
Booting from serial...
Press Q or ESC to abort boot completely.
sLS0dSMmekuro
[LXTERM] Received firmware download request from the device.
[LXTERM] Uploading ./build/firmware.bin to 0x00260000 (29848 bytes)...
[LXTERM] Upload complete (3.7KB/s).
[LXTERM] Booting the device.
[LXTERM] Done.
execute 'run' to start the program

PogoBoot> run
18216,1.0000,1.0000,0,0
18216,1.0000,1.0000,1,0

```

FIGURE 4 – Terminal indiquant que le robot ID = 18216, est celui avec les concentrations initiales à 1.0

```

PogoBoot> run
21803,0.0000,0.0000,0,0
21803,0.0000,0.0000,1,0
21803,0.0000,0.0000,2,0
21803,0.0000,0.0000,3,0
21803,0.0000,0.0000,4,0
21803,0.0000,0.0000,5,0
21803,0.0000,0.0000,6,0
21803,0.0000,0.0000,7,0
21803,0.0000,0.0000,8,0
21803,0.0000,0.0000,9,0
21803,0.0000,0.0000,10,0
21803,0.0000,0.0000,11,0

```

FIGURE 5 – Terminal indiquant que le robot ID = 21803, a des concentrations nulles

```

18216,0.2000,0.2000,6425,87
18216,0.2000,0.2000,6426,87
18216,0.2000,0.2000,6427,87
18216,0.2000,0.2000,6428,87
18216,0.2000,0.2000,6429,87
18216,0.2000,0.2000,6430,87
18216,0.2000,0.2000,6431,87
18216,0.2000,0.2000,6432,87

```

FIGURE 6 – Trace sur le terminal qui indique le consensus

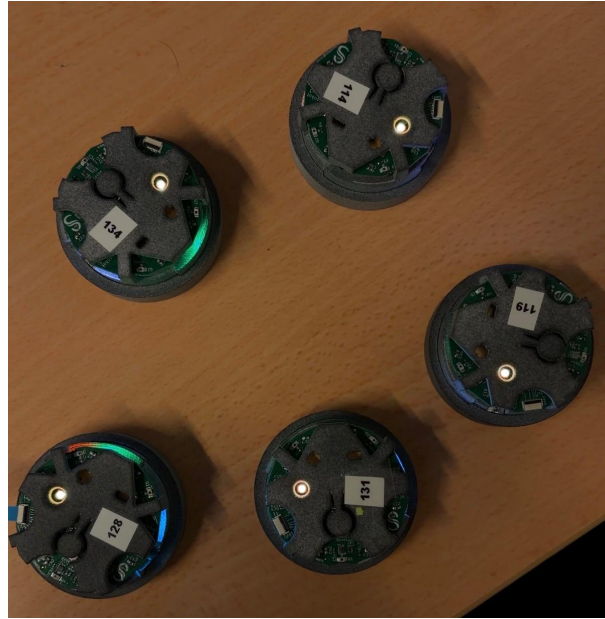


FIGURE 7 – Image des robots qui ont bien atteint le consensus, avec une LED jaune

Sur la figure 6, nous voyons la trace du programme sur le robot qui avait à l'initial 1.0 comme concentrations, et à la suite du programme nous voyons que cette valeur à atteint 0.2 (ce qui correspond bien à la valeur attendue $\frac{\text{concentration}}{\text{nombre}_{\text{robots}}} = \frac{1}{5} = 0.2$). En effet, on peut voir que les deux diffusions en parallèle ont réussi à atteindre un consensus, avec la même valeur. Ce qui fait de ses résultats des estimations plutôt fiables.

Et sur la dernière figure nous voyons que tous les robots ont bien la même couleur de LED (qui est jaune, ce qui correspond bien à celle attendue pour notre valeur de concentration), ce qui indique visuellement que le consensus à bien été atteint.

3.3 Troisième livrable : Mode de diffusion en séquence

Ce livrable vise l'exécution séquentielle d'algorithmes de diffusion. Le système permet d'enchaîner des diffusions successives, ce qui offre une tolérance aux pannes : en cas de suppression ou de défaillance ponctuelle de robots, une nouvelle diffusion peut être relancée périodiquement pour mettre à jour le comptage. Cette approche séquentielle est particulièrement adaptée aux environnements dynamiques où l'état du réseau évolue dans le temps. Elle permet également de garantir une remise à jour régulière des estimations, évitant ainsi une dérive progressive.

3.3.1 Stratégie

Pour cette stratégie, nous avons envisagé qu'après un certain nombre de pas de temps, les robots déclenchent une nouvelle diffusion. La difficulté principale a été de synchroniser l'ensemble des robots sur le début d'une nouvelle diffusion. En effet, il était possible que certains robots soient déjà à la diffusion $n + 1$ pendant que d'autres en soient encore à la diffusion n . Cette désynchronisation rendait complexe le partage des nouvelles informations.

Pour résoudre ce problème, nous avons mis en place une solution basée sur un compteur de diffusion, partagé entre les robots. Chaque robot transmet son numéro de diffusion courant aux autres via les messages. Ainsi, un robot encore à la diffusion n peut détecter qu'un voisin est passé à la diffusion $n + 1$ et basculer à son tour. Cette stratégie repose donc principalement sur la communication entre les robots.

3.3.2 Mise en place (modifications du code)

- Pour mettre en place cette stratégie, nous avons procédé aux modifications suivantes :
- Ajout d'un entier `diffusion_ind` à la structure du robot ainsi qu'à celle des messages. Cet entier définit quelle diffusion est en cours pour chaque robot et peut être transmis dans les messages.
 - Introduction d'un compteur `last_diffusion_seen` pour déterminer si un robot doit basculer vers une autre diffusion. Celui-ci sert à comparer le numéro de diffusion du robot avec celui des autres robots, et à décider s'il doit basculer ou non.
 - Mise en place d'un système non aléatoire pour le déclenchement des nouvelles diffusions. Ce système compte le nombre d'échanges effectués (via `index_communication`) et calcule une probabilité de lancer une nouvelle diffusion. Ainsi, plus le nombre de messages échangés est élevé, plus la probabilité de relancer une diffusion est grande.

3.3.3 Résultats

Après avoir testé notre algorithme sur les deux robots dont nous disposions (sur lesquels il fonctionnait correctement), nous avons ensuite effectué des tests sur six robots afin de vérifier le bon fonctionnement de notre code avec un plus grand nombre de robots.



FIGURE 8 – Lancement de rediffusion

```

39726, 0.0498, 5465, 64
39726, 0.0498, 5466, 64
39726, 0.0498, 5467, 64
39726, 0.0498, 5468, 64
39726, 0.0498, 5469, 64
39726, 0.0498, 5470, 64
39726, 0.0498, 5471, 64
39726, 0.0498, 5472, 64
39726, 0.0498, 5473, 64
39726, 0.0498, 5474, 64
39726, 0.0498, 5475, 64
39726, 0.0498, 5476, 64
39726, 0.0498, 5477, 64
39726, 0.0498, 5478, 64
39726, 0.0498, 5479, 64
39726, 0.0498, 5480, 64
-----Relance diffusion 3 (index 26)-----
c=1, 3
39726, 1.0000, 5481, 64
39726, 1.0000, 5482, 64
39726, 1.0000, 5483, 64
39726, 1.0000, 5484, 64
39726, 1.0000, 5485, 64
39726, 1.0000, 5486, 64
39726, 1.0000, 5487, 64
39726, 1.0000, 5488, 64
39726, 1.0000, 5489, 64
39726, 1.0000, 5490, 64
39726, 1.0000, 5491, 64
39726, 1.0000, 5492, 64
39726, 1.0000, 5493, 64
39726, 1.0000, 5494, 64
39726, 1.0000, 5495, 64
39726, 1.0000, 5496, 64
Robot 39726 | diff=3
39726, 0.1447, 5497, 65
39726, 0.1447, 5498, 65
39726, 0.1447, 5499, 65

```

FIGURE 9 – Terminal indiquant une nouvelle diffusion lancée

Nous avons observé le bon déclenchement d'une nouvelle diffusion. Nous avons par ailleurs vérifié que tous les robots étaient synchronisés sur la même diffusion à l'aide des messages affichés sur le terminal. De plus les LEDs du haut nous ont permis aussi de visualiser cet état : elles s'allument en rouge (pour le robot détenant toute la concentration initialement) ou en bleue (pour les autres robots) à chaque lancement d'une nouvelle diffusion.

3.4 Quatrième livrable : Mécanisme de consensus et de convergence

Ce livrable étudie et compare les méthodes de diffusion permettant d'atteindre un consensus au sein du système. Le consensus est considéré comme atteint lorsque l'ensemble des robots partagent une valeur identique (par exemple, la même estimation du nombre total de robots).

Les contributions réalisées sont les suivantes :

- Implémentation d'un mécanisme de détection de consensus appliqué à une diffusion principale (chargée du comptage).
- Conception d'un détecteur de convergence vérifiant que tous les robots ont obtenu la même estimation, avec un seuil de tolérance paramétrable.
- Mise en place d'une diffusion principale robuste permettant aux robots de s'estimer mutuellement.
- Déploiement d'une diffusion secondaire plus rapide, dédiée à la vérification binaire (oui/non) de l'accord collectif, sans surcharge inutile.

Ce double niveau (diffusion lente pour l'estimation, diffusion rapide pour le consensus) offre un bon compromis entre précision et temps de réponse.

3.4.1 Stratégie

Ce quatrième livrable a nécessité une réflexion approfondie, car il s'agissait de permettre aux robots de vérifier leurs concentrations ainsi que celles de leurs voisins, puis de partager un résultat collectif. Concrètement, chaque robot devait pouvoir demander à ses voisins : « *As-tu la même concentration que moi ?* », et ainsi de suite. La réponse attendue était *oui* si le robot voisin avait la même concentration, *non* dans le cas contraire. Cette stratégie s'est avérée complexe à mettre en œuvre.

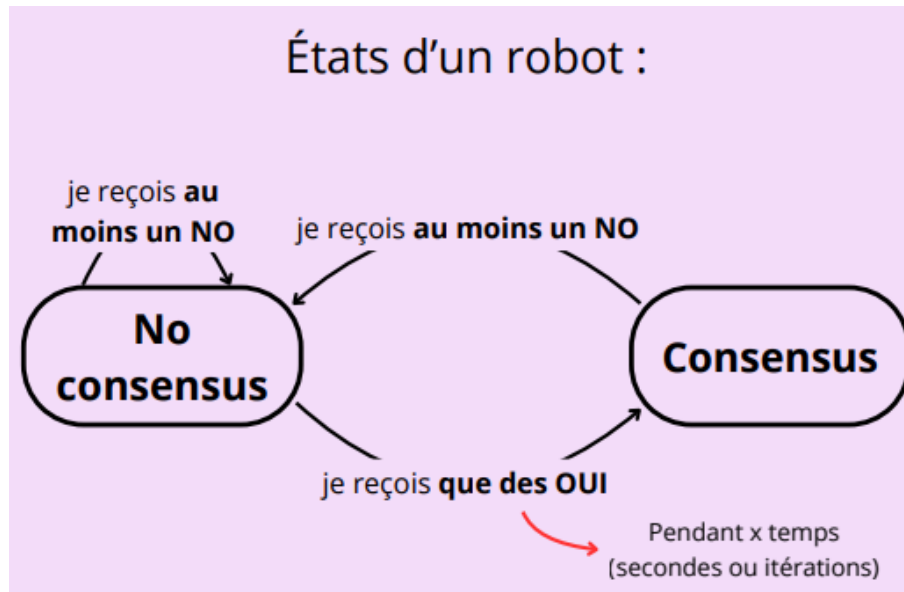


FIGURE 10 – Etats d'un robot

Après plusieurs essais, nous avons finalement abouti à une idée concrète. Dans un premier temps, nous avons implémenté un nouveau type de message, envoyé spécifiquement pour demander si la concentration d'un robot est identique à celle de son voisin. Si l'ensemble des robots répondent *oui*, cela signifie qu'ils sont tous en accord et que le comptage a été correctement réalisé. Dans un second temps, il était nécessaire d'assurer l'obtention d'un consensus. Pour ce faire, nous avons conçu une nouvelle fonction qui, à la fois, compare les concentrations du robot avec celles de ses voisins et comptabilise le nombre d'échanges successifs durant lesquels tous les robots ont répondu *oui*. Ce mécanisme permet de valider que le consensus est bien atteint de manière stable et durable.

3.4.2 Mise en place (modifications du code)

Pour implémenter cette stratégie, nous avons procédé aux modifications suivantes dans le code :

- Création d'un nouveau type de message `CHECK`, dédié à la vérification de l'accord entre robots. Ce message contient notamment la concentration du robot émetteur, son estimation du nombre total de robots (`count_robots`), ainsi qu'un booléen `same_count` indiquant si le robot est déjà en accord avec ses voisins.
- Ajout de champs dans la structure du robot (et dans la structure `connecting_robot`) pour stocker ces nouvelles informations : `count_robots` (valeur estimée du comptage) et `same_count` (état d'accord local).
- Lorsqu'un robot reçoit un message `CHECK`, il compare son propre `count_robots` avec celui de son voisin. Si les deux valeurs sont égales, le booléen `same_count` est mis à `true` pour le robot et pour son partenaire ; sinon, il passe à `false`.
- Mise en place d'une vérification périodique : tous les 10 échanges, chaque robot envoie un message `CHECK` à ses voisins pour s'assurer que l'estimation du nombre de robots est partagée.
- Implémentation d'une fonction `check_consensus_stability` qui détermine si un consensus global est atteint. Cette fonction prend en entrée le consensus local du robot (accord avec ses voisins) et l'estimation courante du nombre de robots. Elle incrémente un compteur de stabilité tant que le robot est localement en accord et que l'estimation ne change pas. Lorsque ce compteur atteint un seuil fixé (300 steps en général), le consensus global est déclaré atteint et une LED des robots passent en rose.

Pseudo-code de la fonction de vérification de consensus :

```
FONCTION check_consensus_stability(neighbor_consensus, *global_consensus,
count_robots):
    SI neighbor_consensus = VRAI ET last_count = count_robots ALORS
        stable_cpt = stable_cpt + 1

        SI stable_cpt >= CONSENSUS_STABLE ET global_consensus = FAUX ALORS
            global_consensus = VRAI
            AFFICHER "CONSENSUS GLOBAL ATTEINT : count_robots"
            //allumer LEDs roses
        FIN SI
    SINON
        stable_cpt = 0    // Reset du compteur
    FIN SI

    last_count = count_robots

FIN FONCTION
```

Cette fonction prend en entrée `neighbor_consensus` un booléen indiquant si le robot a atteint le consensus local avec ses voisins, `global_consensus` un pointeur vers le booléen de consensus global et `count_robots` l'estimation du nombre total de robots dans l'essaim. Et en variables statiques on a `stable_cpt` un compteur de steps consécutifs où le consensus est stable et `last_count` la dernière valeur de `count_robots` enregistrée

3.4.3 Résultats

Nous avons tout d'abord testé notre code sur deux robots.



FIGURE 11 – Consensus atteint pour 2 robots



FIGURE 12 – Consensus atteint pour deux robots, Leds roses

Nous avons ensuite testé notre code sur cinq robots.

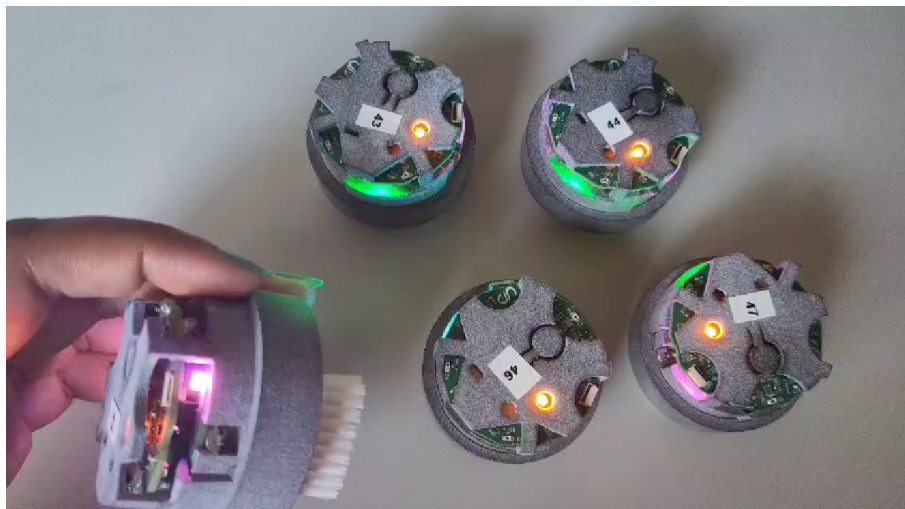


FIGURE 13 – Leds roses apparentes indiquant un consensus

Après un certain temps, les LEDs roses se sont allumées sur l'ensemble des robots, ce qui indique qu'un consensus a été atteint. Les robots ont donc tous réussi à se compter mutuellement et disposent tous de la même valeur.

4 Conclusion

4.1 Bilan

Ce projet nous a permis de découvrir concrètement les problématiques liées à la robotique en essaim et aux systèmes distribués. À travers l'utilisation des robots `Pogobot-wheel`, nous avons pu mettre en œuvre plusieurs mécanismes de diffusion d'information, de synchronisation et de consensus dans un environnement dynamique où les robots communiquent uniquement avec leurs voisins immédiats.

L'un des principaux apports du projet a été la compréhension des difficultés liées aux communications distribuées. En effet, la nature locale des échanges rendent la synchronisation complexe, notamment lors de l'exécution de plusieurs diffusions ou du lancement de nouvelles diffusions en séquence. Nous avons dû adapter et modifier le code existant afin de garantir la cohérence des informations échangées et éviter les désynchronisations entre robots.

Le second livrable nous a permis d'expérimenter la diffusion parallèle d'informations. Nous avons réussi à exécuter plusieurs mécanismes de diffusion simultanément en dupliquant certaines structures et variables du programme. Les résultats obtenus montrent que les robots convergent correctement vers une estimation commune malgré la coexistence de plusieurs flux d'information.

Le troisième livrable a mis en évidence l'importance de la synchronisation dans un système distribué. Grâce à l'ajout d'un identifiant de diffusion partagé entre les robots, nous avons pu garantir que tous les robots participent à la même diffusion au même moment. Les expérimentations réalisées sur plusieurs robots ont confirmé le bon fonctionnement du mécanisme de rediffusion.

Enfin, le quatrième livrable nous a permis d'implémenter un mécanisme de consensus distribué. Les robots sont capables de comparer leurs estimations locales avec celles de leurs voisins et de vérifier collectivement qu'ils possèdent tous la même valeur. L'utilisation des LEDs comme retour visuel nous a permis de suivre facilement l'évolution du système et de confirmer l'atteinte du consensus.

4.2 Limites

Malgré les résultats obtenus, plusieurs limites expérimentales et techniques ont été rencontrées au cours du projet. La principale contrainte concerne le nombre réduit de robots sur lequel nous avons testé nos algorithmes, ce qui a limité l'évaluation des algorithmes sur des essaims de grande taille. Certaines expérimentations n'ont donc pu être réalisées que sur un faible nombre de robots, ce qui réduit la portée statistique des résultats obtenus.

Les communications infrarouges constituent également une source de limitations. La portée réduite des échanges ainsi que la sensibilité aux positions relatives des robots rendent les communications parfois instables.

4.3 Perspectives

Ce projet nous a également permis d'acquérir des compétences importantes en programmation embarquée, en communication entre agents autonomes ainsi qu'en conception d'algorithmes distribués. Plusieurs perspectives d'amélioration peuvent désormais être envisagées.

Une première perspective consisterait à augmenter le nombre de robots utilisés afin d'étudier le comportement des algorithmes sur des essais plus importants. Cela permettrait notamment d'analyser plus précisément les temps de convergence et la robustesse des mécanismes de consensus.

Il serait également intéressant d'étudier des scénarios plus dynamiques, dans lesquels les robots se déplaceraient. Dans ce contexte, les voisinages changeraient continuellement et les algorithmes devraient s'adapter à des topologies de communication fortement variables.

D'autres améliorations pourraient porter sur l'optimisation des mécanismes de diffusion afin de réduire le nombre de messages échangés et d'accélérer la convergence. Enfin, une perspective importante serait d'améliorer la robustesse du système face aux défaillances de robots ou aux pertes de communication, afin de rendre les comportements collectifs plus fiables dans des environnements réels.

5 Annexes

5.1 Cahier des charges

5.1.1 Contexte et définition du problème

Le projet s'inscrit dans le cadre de l'évaluation de l'UE PAI2D du master AI2D à Sorbonne Université. Il porte sur la programmation de comportements collectifs pour les robots mobiles `Pogobot-wheel`, des robots de petite taille communiquant par infrarouge avec leurs voisins immédiats.

L'objectif général est d'étudier et d'implémenter des mécanismes distribués de diffusion d'information, de comptage, de synchronisation et de consensus au sein d'un essaim de robots autonomes.

5.1.2 Objectifs fonctionnels

Les objectifs suivants ont été identifiés et répartis en livrables :

1. **Livable 1 – Reproduction d'expériences** Reproduire des expériences existantes sur robots mobiles en faisant varier la répartition initiale des concentrations et le nombre d'itérations, afin d'analyser la convergence du système.
2. **Livable 2 – Diffusion en parallèle** Permettre l'exécution simultanée de plusieurs algorithmes de diffusion (avec des paramètres D différents) sans interférence, et évaluer l'impact sur la convergence.
3. **Livable 3 – Diffusion en séquence** Enchaîner plusieurs diffusions successives de manière synchronisée, avec tolérance aux pannes et mise à jour périodique des estimations.
4. **Livable 4 – Consensus et convergence** Implémenter un mécanisme distribué permettant aux robots de vérifier collectivement qu'ils partagent la même estimation, et de stabiliser cet accord dans le temps.

5.1.3 Contraintes techniques et matérielles

— **Matériel** : robots `Pogobot-wheel`.

- **Communication** : infrarouge, uniquement entre voisins immédiats (portée limitée).
- **Programmation** : langage C, environnement de développement propriétaire des robots, compilation avec `make`, téléversement via `serialboot`.
- **Retour utilisateur** : LEDs bicolores pour visualiser l'état interne des robots (concentration, consensus...).
- **Déploiement** : programmation filaire (un robot à la fois), ou programmation via une "douche" pour plusieurs robots disponible seulement au laboratoire.

5.1.4 Critères d'acceptation

Chaque livrable est considéré comme validé si :

- Les robots exécutent le comportement attendu sans crash ni blocage.
- Les valeurs affichées dans les traces terminal sont cohérentes avec la théorie.
- La couleur des LEDs permet de suivre correctement l'état du système.
- Pour le livrable 4 : un consensus global est atteint (LEDs roses sur tous les robots).

5.1.5 Livrables attendus (code)

- Code source complet et commenté (dépôt Git).
- Fichier `README` avec instructions de compilation et d'exécution.
- Accès au dépôt Git.

5.1.6 Environnement d'exécution

- Ordinateur sous Linux (ou machine du laboratoire `isir_kilobots`).
- Connexion USB → carte de programmation → robot.
- Commande typique : `make connect TTY=/dev/ttyUSBX`.

5.2 Manuel d'utilisation

Comment mettre le code sur un robot ?

- Aller sur le dossier du livrable souhaité : livrable-1 (où il n'y a aucune modification), livrable-2, livrable-3 ou livrable-4.
- Compiler le code avec `make clean all`.
- Appuyez sur le bouton en haut du robot, ceci va permettre de le "reset", et donc d'effacer tout code précédemment téléverser.
- La connexion par câble permet de programmer un seul robot à la fois. Le robot est connecté à la carte de programmation avec un câble ruban plat, et la carte de programmation se connecte à votre machine sur un port USB.



FIGURE 14 – Branchement du robot

- Connectez un robot à votre ordinateur et, dans votre terminal, exécutez cette commande : `make connect TTY=/dev/ttyUSBX` — remplacez X par le numéro de port USB correct.
- Appuyez sur la touche **Entrée** pour accéder à l'invite.
- Téléversez le programme à l'aide de la commande `serialboot`.
- Utilisez la commande `run` pour lancer le dernier programme téléversé.
- Pour quitter, vous devez appuyer sur `Ctrl+C` `Ctrl+C`.

EXEMPLE :

```
(pogobot-venv) noriane@norianePC:~/pogobot-sdk/experiments/size/test/pogobot-size-estimation/livable-4$ make clean all
rm -f -r ./build.*
mkdir -p ./build
CC      build/main.o
CC      build/firmware.elf
/usr/lib/gcc/riscv64-unknown-elf/13.2.0/../../../../riscv64-unknown-elf/bin/ld: warning: build/firmware.elf has a LOAD segment with RWX permissions
chmod -x build/firmware.elf
OBJCOPY build/firmware.bin
chmod -x build/firmware.bin
(pogobot-venv) noriane@norianePC:~/pogobot-sdk/experiments/size/test/pogobot-size-estimation/livable-4$ make connect TTY=/dev/ttyUSB0
/home/noriane/pogobot-sdk/tools/litex_term.py --serial-boot --kernel ./build/firmware.bin --kernel-addr 0x260800 --safe /dev/ttyUSB0
/home/noriane/pogobot-sdk/tools/litex_term.py:575: DeprecationWarning: setDaemon() is deprecated, set the daemon attribute instead
  self.reader_thread.setDaemon(True)
/home/noriane/pogobot-sdk/tools/litex_term.py:604: DeprecationWarning: setDaemon() is deprecated, set the daemon attribute instead
  self.writer_thread.setDaemon(True)

Pogobot> serialboot
Booting from serial...
Press Q or ESC to abort boot completely.
sl5Dd$Mmkekro
[LXTERM] Received firmware download request from the device.
[LXTERM] Uploading ./build/firmware.bin to 0x0260800 (30328 bytes)...
[LXTERM] Upload complete (3.7KB/s).
[LXTERM] Booting the device.
[LXTERM] Done.
        execute 'run' to start the program

Pogobot> run
39726,1.0000,0.0
39726,1.0000,1.0
39726,1.0000,2.0
39726,1.0000,3.0
39726,1.0000,4.0
39726,1.0000,5.0
39726,1.0000,6.0
39726,1.0000,7.0
39726,1.0000,8.0
39726,1.0000,9.0
39726,1.0000,10.0
39726,1.0000,11.0
39726,1.0000,12.0
```

FIGURE 15 – Terminal